

Sorokin School



Task-Management System [часть 3]

🔥 Задание к модулю №3 по интенсиву Spring Boot

Сегодня вы переходите на новый уровень — вы начнете работать с реальной базой данных! Это важный шаг, который позволит вам понять, как приложения общаются с базами данных и как применять аннотации JPA для удобного взаимодействия с базой.

Сегодня вы не только научитесь работать с `Hibernate` и `PostgreSQL`, но и добавите новый важный функционал — перевод задачи в статус `IN_PROGRESS` с соблюдением бизнес-правил. Это позволит вам увидеть, как масштабируется приложение при добавлении новых возможностей.

Описание

В этом задании вы мигрируете своё приложение с in-memory хранилища на использование реальной базы данных (поднимите PostgreSQL с помощью Docker).

Вы интегрируете Spring Data JPA и Hibernate, сделаете сущность `TaskEntity` как JPA-Entity и реализуете HTTP REST CRUD-операции через репозиторий.

Знания для выполнения (что вам предстоит закрепить)

- Работа с реальной базой данных через Spring Data JPA.
- Настройка подключения к базе данных с использованием Docker.
- Использование аннотаций JPA (`@Entity`, `@Id`, `@GeneratedValue`, `@Table` и т.д.) для описания сущностей.
- Интеграция Hibernate для ORM (Object-Relational Mapping).
- Перенос логики хранения данных из in-memory слоя в репозиторий, работающий с базой.

Задание на разработку

🔥 Что конкретно детально нужно реализовать

Добавление зависимости PostgreSQL

Добавьте в ваш проект зависимости для PostgreSQL драйвера и spring data JPA

```
<?xml version="1.0"?>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

Настройте параметры подключения к базе данных в `application.properties` (или `application.yml`).

Миграция сущности в Hibernate

- Создайте новый класс `TaskEntity`.
- Аннотируйте его с помощью `@Entity` и `@Table` для задания имени таблицы.
- Перенесите в `TaskEntity` все необходимые поля (`id`, `creatorId`, `assignedUserId`, `status`, `createDateTime`, `deadlineDate`, `priority`) и примените аннотации:
 - `@Id` и `@GeneratedValue` для поля `id`.
 - `@Column` для всех остальных полей для указания имени столба в таблице
- Обновите настройки вашего сервиса так, чтобы CRUD-операции выполнялись через репозиторий, работающий с `TaskEntity`.

Создание репозитория:

- Создайте интерфейс `TaskRepository`, который расширяет `JpaRepository<TaskEntity, Long>`.
- Это позволит вам получать базовый функционал для работы с базой данных.

Перенос CRUD-операций в сервис:

- Обновите класс `TaskService`, чтобы операции создания, получения, обновления и удаления задач выполнялись через `TaskRepository`.
- При этом сохраните бизнес-логику, которую вы реализовывали ранее, но теперь для работы с `TaskEntity`.

Добавление нового эндпойнта для перевода задачи в IN_PROGRESS:

- В классе `TaskController` создайте новый метод, обрабатывающий HTTP-запрос:
 - `POST /tasks/{id}/start`
- Бизнес-логика в сервисе при вызове этого эндпойнта должна включать:
 - Проверку, что поле `assignedUserId` заполнено. Если нет — возвращайте ошибку с понятным сообщением.
 - Проверку, что у пользователя с заданным `assignedUserId` в данный момент не более 4 активных задач в статусе `IN_PROGRESS` (чтобы общее число не превышало 5). Если ограничение нарушается — возвращайте ошибку.
 - При успешном выполнении бизнес-логики, перевод задачи в статус `IN_PROGRESS`.
- Обеспечьте корректное использование HTTP-статусов (например, 200 OK при успешном переводе или 404 Not Found при ошибках).

Советы по реализации

Немного заметок о том как лучше реализовать задание

Проверка подключения к базе:

- Перед внесением изменений убедитесь, что ваша база данных запущена через Docker и параметры подключения правильно настроены.

```
application.properties
spring.application.name=task-system
```